

Projekt: Software Engineering

Architekturdokument

MietManager

Portfolio von Michael Bangas

Matrikelnummer: IU14132885

Kursbezeichnung: Projekt: Software Engineering (DLMCSPSE01_D)

Studiengang: M.Sc. Informatik

IU Internationale Hochschule

25. September 2026

Inhaltsverzeichnis

1	Technologieübersicht	1
1.1	Eingesetzte Technologien	1
1.2	Betrachtete Alternativen	1
2	Architekturübersicht	2
3	Struktur	3
4	Verhalten	4
5	Fazit	5

Abbildungsverzeichnis

2.1	Schichtenarchitektur von MietManager	3
3.1	UML-Klassendiagramm der Geschäftsobjekte von MietManager.	4
4.1	UML-Sequenzdiagramm zum Prozess der Erstellung und Export der Nebenkostenabrechnung.	5

1 Technologieübersicht

Dieser Abschnitt beschreibt zunächst die eingesetzten Technologien und Werkzeuge von MietManager und stellt anschließend die jeweils betrachteten Alternativen sowie die Gründe für ihre Ablehnung dar.

1.1 Eingesetzte Technologien

MietManager ist in Python 3.14 implementiert, das durch seine breite Standardbibliothek, die kurze Entwicklungszeit durch dynamische Typisierung und die gute Verfügbarkeit ausgereifter Pakete für alle benötigten Teilbereiche wie GUI, PDF-Erzeugung und Tests für ein zeitlich auf vier Wochen begrenztes Portfolioprojekt geeignet ist. Für die grafische Oberfläche kommt PyQt6 zum Einsatz, das unter Windows ein natives Erscheinungsbild, ein umfangreiches Widget-Set sowie mit Qt Style Sheets eine deklarative Möglichkeit zur einheitlichen Gestaltung der Oberfläche bietet. Die Datenhaltung erfolgt über SQLAlchemy 2.0, das die Geschäftsobjekte auf Tabellen einer lokalen SQLite-Datenbank abbildet. SQLite kommt dabei dateibasiert ohne separate Serverinstallation aus, was zur Zielgruppe einer lokalen Single-User-Desktop-Anwendung ohne Netzwerkanbindung passt. Für den PDF-Export der Nebenkostenabrechnung wird ReportLab verwendet, das die Geschäftsbriefe direkt programmatisch erzeugt. Die Geschäftsregeln und die Berechnungslogik werden mit pytest getestet, das flache Testfunktionen statt Testklassen ermöglicht und die Service-Schicht unabhängig von Qt und Datenbank rein in Python prüfbar macht. Mit coverage.py wird zusätzlich gemessen, wie vollständig diese Tests die Service-Schicht tatsächlich abdecken. Für die Auslieferung an Endanwender ohne installierten Python-Interpreter ist PyInstaller vorgesehen, das die Anwendung in eine eigenständige Windows-.exe überführt und damit die im Anforderungsdokument definierte Portabilitätsanforderung erfüllt.

1.2 Betrachtete Alternativen

Als Alternative zu Python kämen die im Rahmen dieses Kurses ebenfalls zugelassenen Sprachen Java, C++ oder C# infrage. Diese hätten zwar eine strengere statische Typisierung geboten, doch der damit verbundene höhere Boilerplate-Aufwand erschien angesichts der knappen Projektdauer nicht gerechtfertigt, weshalb die Wahl auf Python fiel.

Für die grafische Oberfläche stand PyQt6 der in der Python-Standardbibliothek enthaltenen Alternative Tkinter gegenüber. Tkinter käme ohne zusätzliche Abhängigkeit aus, produziert unter Windows jedoch ein wenig natives Erscheinungsbild und bietet kein deklaratives Stylesheet-Konzept für eine einheitliche Gestaltung mehrerer Tabs und Dialoge, weshalb die Wahl auf PyQt6 fiel.

Im Vergleich zu einer Client-Server-Datenbank wie PostgreSQL oder MySQL vermeidet die dateiba-

sierte SQLite-Datenbank den zusätzlichen Installations- und Wartungsaufwand eines Datenbankservers, den PostgreSQL oder MySQL erfordert hätten, ohne dass die Zielgruppe der lokalen Single-User-Anwendung auf dessen Fähigkeiten angewiesen wäre.

Für den PDF-Export wurde ReportLab HTML-basierten Alternativen wie WeasyPrint vorgezogen, da es die Geschäftsbriefe direkt programmatisch erzeugt, statt den Umweg über eine zusätzliche Templating-Schicht zu gehen. Für die Tests wurde pytest dem in der Standardbibliothek enthaltenen unittest vorgezogen, da pytest flache Testfunktionen statt Testklassen ermöglicht.

2 Architekturübersicht

MietManager folgt einer klassischen Drei-Schichten-Architektur, die Benutzeroberfläche, Geschäftslogik und Datenhaltung strikt voneinander trennt. Alternative Architekturmuster wie eine Microservices-Architektur oder eine Client-Server-Architektur mit eigenständigem Backend-Dienst wurden verworfen, da MietManager als lokale Single-User-Desktop-Anwendung ohne Netzwerkanbindung betrieben wird und die mit einer Aufteilung in unabhängig deploybare Dienste verbundene zusätzliche Komplexität, etwa durch Interprozesskommunikation oder Serviceorchestrierung, hier keinen Mehrwert böte. Eine Schichtenarchitektur erlaubt stattdessen bereits innerhalb eines einzelnen Prozesses die für die im Testdokument beschriebene isolierte Testbarkeit der Geschäftslogik nötige Trennung, ohne die Betriebskomplexität eines verteilten Systems einzuführen.

Die UI-Schicht im Paket `mietmanager.ui` besteht aus dem Hauptfenster, den tabellarischen Tabs sowie den Erfassungsdialogen. Sie reagiert auf Benutzerinteraktion, ruft die darunterliegende Serviceschicht auf und stellt deren Ergebnisse bzw. Fehlermeldungen dar, enthält jedoch selbst keine Geschäftslogik. Die Serviceschicht im Paket `mietmanager.services` kapselt die Geschäftsregeln, die Berechnung der Nebenkostenabrechnung sowie den PDF-Export und ist vollständig unabhängig von PyQt6 und damit ohne laufende Anwendung testbar. Die Datenzugriffsschicht in den Paketen `mietmanager.data` und `mietmanager.models` kapselt den Zugriff auf die SQLite-Datenbank über SQLAlchemy-Modelle, Engine und Session. Abbildung 2.1 visualisiert die drei Schichten und ihre Abhängigkeitsrichtung. Aufrufe verlaufen ausschließlich von der UI- über die Service- zur Datenzugriffsschicht, nie umgekehrt, sodass die im Spezifikationsdokument definierten Geschäftsregeln unabhängig von der Oberfläche gelten.

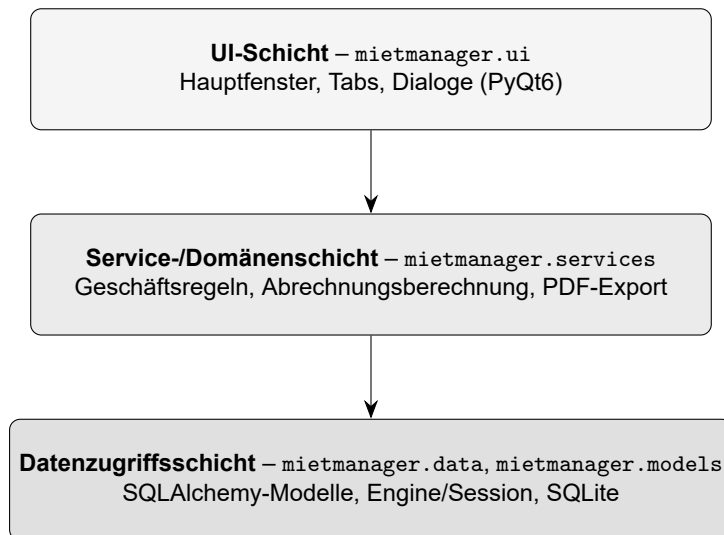


Abbildung 2.1: Schichtenarchitektur von MietManager

3 Struktur

Die Struktur von MietManager wird über dieselben acht Geschäftsobjekte beschrieben, die bereits im Spezifikationsdokument als UML-Klassendiagramm eingeführt wurden. Es handelt sich um Immobilie, Mieteinheit, Mieter, Mietvertrag, Kostenposition, Nebenkostenabrechnung, Abrechnungsposition und Vermieterprofil. Während das Klassendiagramm im Spezifikationsdokument den fachlichen Zusammenhang der Geschäftsobjekte im Vordergrund stellt, zeigt Abbildung 3.1 dieselben Klassen zusätzlich mit ihren wichtigsten Attributen und Methoden, wie sie in `models/entities.py` als SQLAlchemy-Modellklassen implementiert sind. Über die reine Datenhaltung hinaus besitzt von den acht Klassen nur `Vermieterprofil` eigenes Verhalten. Die Methode `is_complete()` prüft, ob Name, Straße, Postleitzahl und Ort gesetzt sind, und wird von der Service-Schicht vor dem PDF-Export aufgerufen. Dass die Geschäftsobjekte insgesamt so wenige Methoden besitzen, ist eine bewusste Folge der in Abschnitt 2 beschriebenen Schichtenarchitektur. Die Modellklassen in `models/entities.py` sind reine SQLAlchemy-Datenklassen ohne Geschäftslogik, während sämtliche Berechnungen und Regeln in der Service-Schicht als eigenständige Funktionen liegen, die Modellobjekte lediglich als Parameter entgegennehmen und mit dem Ergebnis befüllen. So besitzen beispielsweise weder `Nebenkostenabrechnung` noch `Abrechnungsposition` eine Berechnungsmethode. Die komplette Berechnungslogik der Nebenkostenabrechnung steckt stattdessen in der Funktion `erstelle_abrechnung(session, immobilie, start, ende)` aus `services/abrechnung.py`, die in Abbildung 4.1 als Aufruf von `AbrechnungTab` an die Service-Schicht dargestellt ist. Diese Trennung hält die Modellklassen schlank und stellt sicher, dass Geschäftslogik wie in Abschnitt 2 gefordert unabhängig von Qt und Datenbank testbar bleibt.

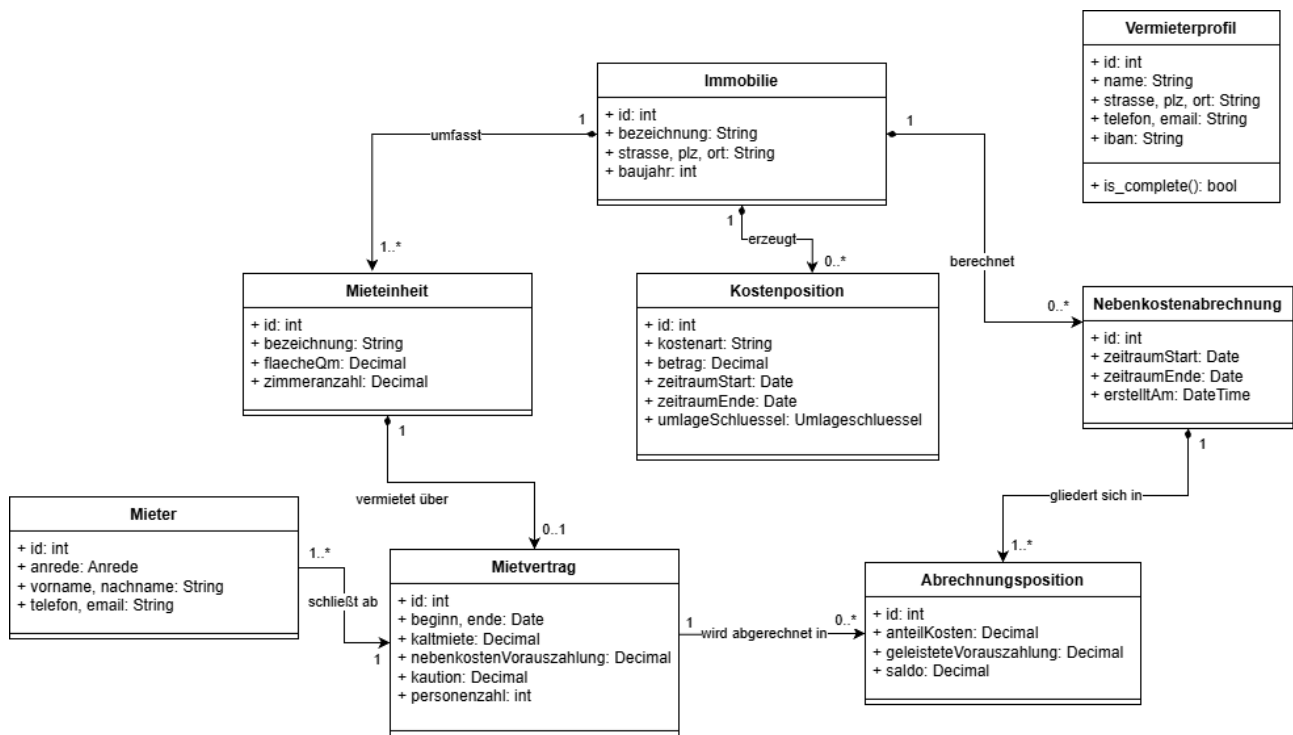


Abbildung 3.1: UML-Klassendiagramm der Geschäftsobjekte von MietManager.

4 Verhalten

Als zentraler Systemablauf wird der Kernprozess „Nebenkostenabrechnung erstellen und als PDF exportieren“ betrachtet, der bereits im Spezifikationsdokument fachlich als UML-Aktivitätsdiagramm beschrieben ist und in Abbildung 4.1 technisch als UML-Sequenzdiagramm mit den tatsächlichen Funktionsaufrufen über die Schichten hinweg dargestellt wird. Jeder durchgängige Hin-Pfeil entspricht dabei einem konkreten Aufruf aus dem Quellcode. Besitzt der Aufruf einen Rückgabewert, zeigt der zugehörige gestrichelte Rückgabe-Pfeil die Zuweisung an die im Code tatsächlich verwendete Variable, z. B. `dateien = exportiere_abrechnung(...)`. Die schmalen Balken auf den Lebenslinien markieren die Aktivierungsdauer. Die drei Service-Module `abrechnung.py`, `regeln.py` und `pdf_export.py` sind dabei als eigene Lebenslinien dargestellt, statt sie wie in Abschnitt 2 zur Serviceschicht zusammenzufassen, damit erkennbar bleibt, welcher Aufruf tatsächlich in welchem Modul landet. Der Vermieter löst über den Klick auf „Abrechnung erstellen“ die gleichnamige Methode auf `AbrechnungTab` aus, die ihrerseits die Berechnungsfunktion `erstelle_abrechnung` aus `services.abrechnung` aufruft. Diese persistiert das Ergebnis über die SQLAlchemy-Session und gibt das `Nebenkostenabrechnung`-Objekt zurück. Beim anschließenden PDF-Export prüft `AbrechnungTab` zunächst per `pruefe_profil_vollstaendig` aus `services.regeln`, ob das Vermieterprofil vollständig ist. Anschließend wird `exportiere_abrechnung` aus `services.pdf_export` aufgerufen und je Mietvertrag ein PDF-Brief in das gewählte Zielverzeichnis geschrieben.

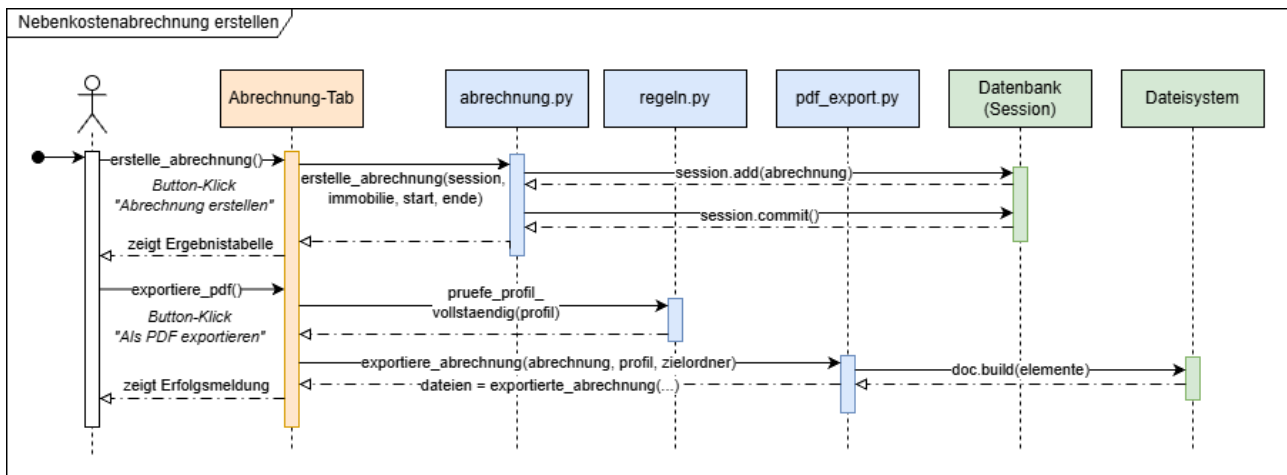


Abbildung 4.1: UML-Sequenzdiagramm zum Prozess der Erstellung und Export der Nebenkostenabrechnung.

5 Fazit

Dieses Dokument hat die technische Umsetzung der im Spezifikationsdokument fachlich beschriebenen Anforderungen dargestellt. Dazu zählen die eingesetzten Technologien, die Drei-Schichten-Architektur aus UI-, Service- und Datenzugriffsschicht, die acht Geschäftsobjekte als schlanke SQLAlchemy-Modellklassen sowie der Kernprozess „Nebenkostenabrechnung erstellen und als PDF exportieren“ als UML-Sequenzdiagramm mit den tatsächlichen Funktionsaufrufen über die Schichten hinweg. Auf dieser technischen Grundlage wird die Anwendung implementiert und anschließend im Testdokument der Finalisierungsphase gegen Unit- und Systemtests verifiziert.